Implementación de Algoritmo *Flood Fill* para Navegación Autónoma en Software Embebido

D. N. Mozo, S. N. Garré, I. P. Lacoste Larrieu, A. Loyza y F. Iampietro
Depto. de Electrónica y Computación, Facultad de Ingeniería, UNMDP
Mar del Plata, 7600, Argentina
{dmozo, sgarre, ianlarrieu, augustoloyza, iampietro}@alumnos.fi.mdp.edu.ar

***Resumen*—En este trabajo se presenta la implementación de un sistema de control digital sobre un microcontrolador de 32 bits basado en un núcleo ARM Cortex-M4, destinado a un robot móvil de dos ruedas capaz de desplazarse de forma autónoma, detectar obstáculos y encontrar el camino mínimo hacia un objetivo. La navegación se realiza mediante el algoritmo *flood fill*, que permite construir un mapa del entorno y determinar la ruta óptima hacia la meta. El desarrollo se implementó íntegramente en lenguaje C, logrando un comportamiento eficiente, estable y en tiempo real del sistema.**

***Palabras clave*—STM32, C, Robot, Maze-solver, Flood-fill.**

I. Introducción

La resolución de laberintos es un problema clásico en robótica móvil y teoría de grafos, que consiste en encontrar una ruta eficiente desde una posición inicial hasta un destino dentro de un entorno desconocido o parcialmente conocido. Para ello, el robot debe detectar obstáculos, construir un mapa del entorno y seleccionar una trayectoria óptima, generalmente bajo criterios de mínima distancia o tiempo. Este tipo de desafíos resulta especialmente relevante en aplicaciones de robótica educativa, navegación autónoma y sistemas logísticos de bajo costo.

Aunque muchas implementaciones se basan en microcontroladores de recursos limitados como el Arduino Uno [1]–[3], en este trabajo se emplea un microcontrolador STM32 con núcleo ARM Cortex-M4, que ofrece mayor potencia de cómputo y mejores prestaciones para el control de sensores y actuadores. Esto permite ejecutar en tiempo real el algoritmo de navegación *flood fill* sobre un robot móvil diferencial de dos ruedas, logrando una movilidad autónoma más eficiente y precisa. El sistema implementa una versión del algoritmo *flood fill* mediante estructuras de datos almacenadas en memoria RAM, procesadas secuencialmente por el microcontrolador. El laberinto se modela como una matriz que se actualiza dinámicamente a medida que el robot se desplaza y detecta nuevos obstáculos. El proyecto fue desarrollado en el marco de la asignatura Sistemas Digitales (4.º año de Ingeniería Electrónica), en la cual se introducen contenidos de diseño digital, programación de microcontroladores y desarrollo en lenguaje C. El sistema se implementó sobre una placa STM32F407G-DISC1, incorporando un microcontrolador de 32 bits con arquitectura ARM Cortex-M4, lo que permitió alcanzar un desempeño estable y en tiempo real.

II. Algoritmo *flood fill*

El algoritmo de llenado utilizado se basa en una estrategia en la que el robot parte del supuesto de un entorno libre de obstáculos. A medida que avanza, actualiza dinámicamente un mapa interno, asignando a cada celda un valor que representa la distancia al objetivo y marcando como inaccesibles aquellas celdas adyacentes a los muros detectados por sus sensores. Esta información se emplea para recalcular rutas óptimas, permitiendo que el robot se adapte progresivamente al entorno real y construya su propio mapa del laberinto [3]. El algoritmo fue implementado inicialmente para un laberinto de 4×4 celdas, utilizado en las pruebas y demostraciones. Sin embargo, la estructura del código emplea dos bucles anidados (`for`), lo que permite escalar fácilmente a cualquier tamaño de laberinto cuadrado sin modificar la lógica central.

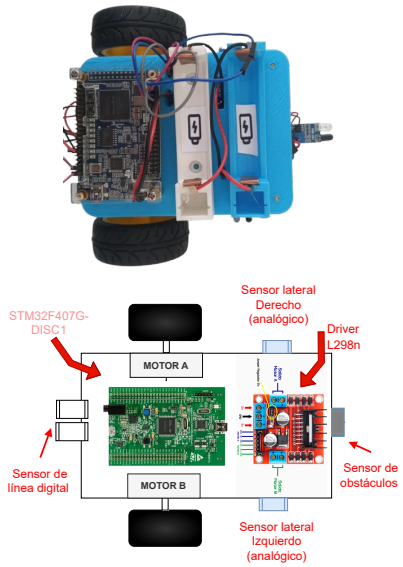


Figura 1. Esquema funcional del robot implementado.

III. Implementación

III-A. Robot

El robot de la Fig. 1 incorpora dos motores de corriente continua controlados por un driver L298N [4]. Además, dispone de un sensor frontal digital, un sensor digital para detección de línea, y dos sensores laterales infrarrojos analógicos.

El control y procesamiento de los datos de los sensores, así como el accionamiento de los motores, se gestionan en la placa STM32F407G-DISC1 [5]. El microcontrolador lee los valores analógicos de los sensores laterales mediante su ADC interno, y procesa esta información para mantener un desplazamiento recto y estable.

Se configuraron pines GPIO en modo entrada (botón de usuario, sensor de línea) y en modo salida (señales de control del driver de motores). Los eventos asíncronos se gestionan con líneas EXTI e interrupciones priorizadas en el NVIC, permitiendo reaccionar al instante ante la pulsación del botón o cambios críticos de los sensores sin bloquear el ciclo principal.

Las señales analógicas de los sensores infrarrojos laterales se adquieren con el ADC interno en modo de escaneo y transferencia por DMA hacia un búfer circular, y luego se promedian los valores para garantizar muestreos estables.

Además, se implementó un mecanismo de **autocalibración** de los sensores infrarrojos de proximidad. Antes de iniciar la exploración del laberinto, durante unos segundos el sistema adquiere de manera continua los valores de cada sensor en reposo. Con esta información se establecen los umbrales de detección, mejorando la robustez frente a variaciones de luz ambiental o diferencias entre sensores.

Para telemetría se integra un módulo Bluetooth con interfaz UART que transmite posición, orientación y estados internos. La recepción y monitoreo se realizaron desde un teléfono móvil utilizando la aplicación *Serial Bluetooth Terminal* [6].

III-B. Laberinto

El laberinto de la Fig. 2 está construido en madera, con columnas impresas en 3D, y diseñado con cuadrículas delimitadas por líneas negras detectables por el robot. Además, cada cuadrícula permite la incorporación de muros, lo que facilita variar la complejidad del entorno en las pruebas.



Figura 2. Laberinto implementado con paredes rebatibles.

IV. Control del Robot

El algoritmo de llenado se implementó en C utilizando arreglos bidimensionales para representar el laberinto. Cada elemento de la matriz almacena la información de muros y su valor de peso (distancia estimada al objetivo), inicialmente las distancias sin considerar ningún muro.

En cada iteración, el software:

1. Lee los sensores y actualiza la información de muros.
2. Recalcula los pesos en la matriz en dodne cada casilla toma el valor del mínimo vecino más uno.
3. Selecciona el siguiente movimiento (avanzar o girar) en función de la celda adyacente accesible con menor peso.

El control de movimiento se implementó mediante temporizadores (TIM) y salidas PWM para regular la velocidad de los motores. Las acciones de los motores se ejecutan con base en los valores leídos por los sensores laterales y el sensor de línea.

Además, se implementó un *modo sprint* activado mediante un pulsador conectado a una línea EXTI. Al presionarlo, el sistema reinicia únicamente la posición del robot a $(0, 0)$, conservando en RAM los pesos y la información de muros aprendidos durante la exploración previa. Al mismo tiempo, se incrementa la consigna de velocidad ajustando los parámetros de PWM, de modo que el robot repite la resolución del laberinto a mayor rapidez aprovechando el mapa ya construido.

IV-A. Variantes de implementación

Durante el desarrollo, se implementaron dos variantes funcionales del algoritmo *flood fill*. Ambas parten de la misma estructura de datos y lógica general, pero difieren en el flujo de ejecución y en la forma de gestionar ciertas decisiones durante la navegación.

- **Primera variante:** prioriza la simplicidad en el control secuencial. Se utilizan interrupciones cuando se detecta un obstáculo o un cambio de cuadrícula. El cálculo de los pesos y la elección del siguiente movimiento se realizan en ciclos completos, reduciendo la complejidad del flujo pero con un tiempo de respuesta ligeramente mayor. Además, el modelado del laberinto está planteado de forma escalable a dimensiones arbitrarias $n \times n$, aunque en esta implementación se probó únicamente con un entorno de 4×4 [7].
- **Segunda variante:** un código básico y robusto con control secuencial. Se utiliza únicamente una interrupción para detectar las líneas de cambio de cuadricula. La resolución del sistema *flood fill* fue hecho de una forma sencilla para resolver laberinto de 4x4. Se empleó la comunicación UART para ver si el funcionamiento es correcto en tiempo real y que mande mensajes en ciertas circunstancias. Se ha implementado una prioridad para ir derecho y evitar giros ya que estos no eran ideales. Debido a eso, también se incorporó un sistema de memoria de la última corrección de modo que haya dos tiempos de giro (si previamente corrigió hacia la izquierda y debe girar a la izquierda el tiempo será menor, y viceversa). Para facilitar la movilidad del robot, se implementó que luego de chocar una pared, retroceda un poco para realizar el giro en el centro de la casilla. Además, después de cruzar una linea, se incluyó un ”tiempo ciego.[en] donde ignore las paredes laterales para evitar correcciones indebidas. Para corregir desvíos se realizó una corrección automática luego de cierto tiempo definido: si no detectó linea o pared corrige al lado opuesto de la última vez. Por último, se realizó que al llegar a la meta, dé un giro 180° y vuelva a la salida (con un vector que almacena las posiciones del camino realizado) [8].

Los diagramas de flujo de ambas variantes se muestran en las Fig. 3 y Fig. 4, donde se evidencian las diferencias en la secuencia de ejecución.

Funciones del sistema de control

- **Desplazamiento centrado y control de motores:** ajuste de velocidad mediante PWM, con lectura constante de

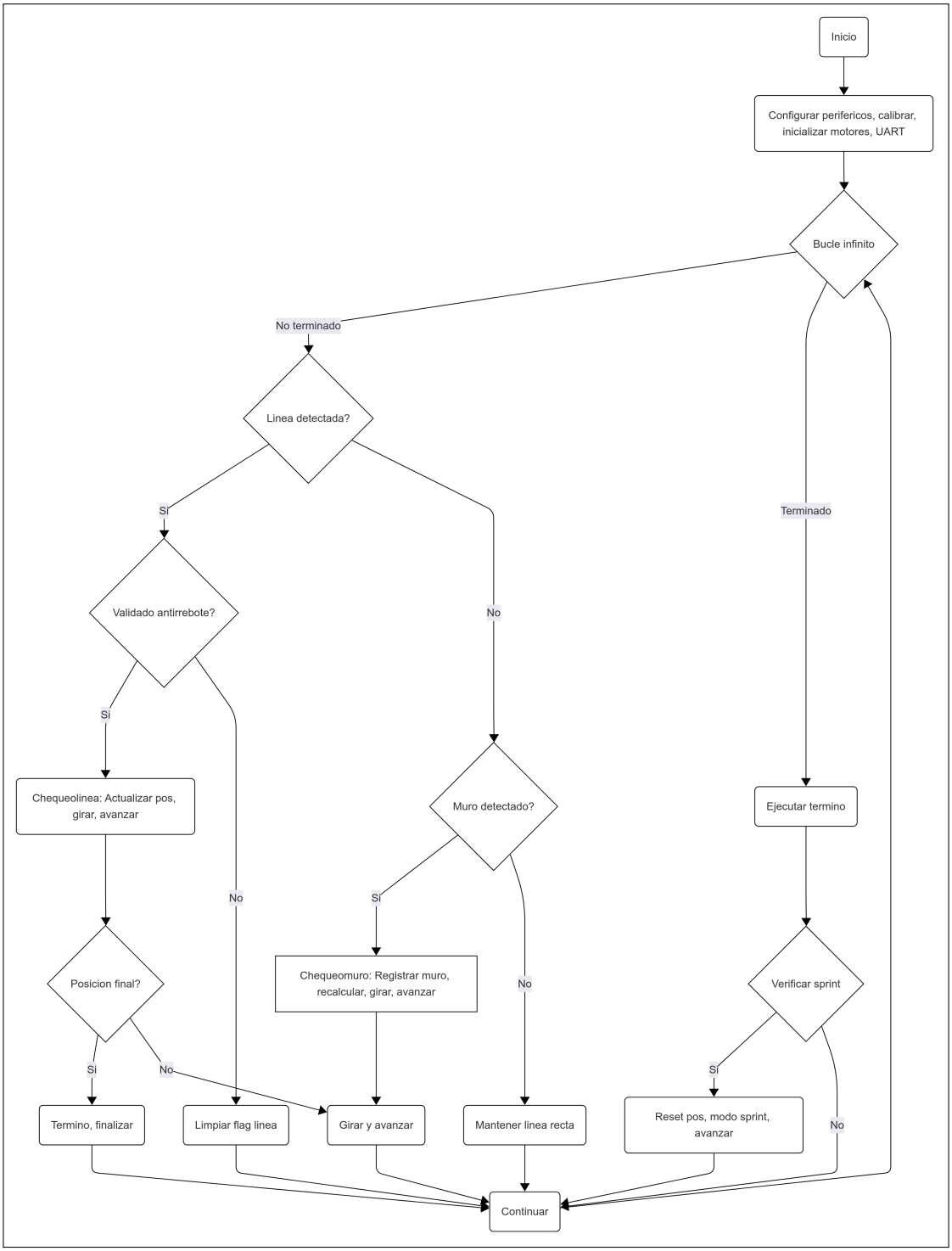


Figura 3. Diagrama de flujo de la primera variante del algoritmo.

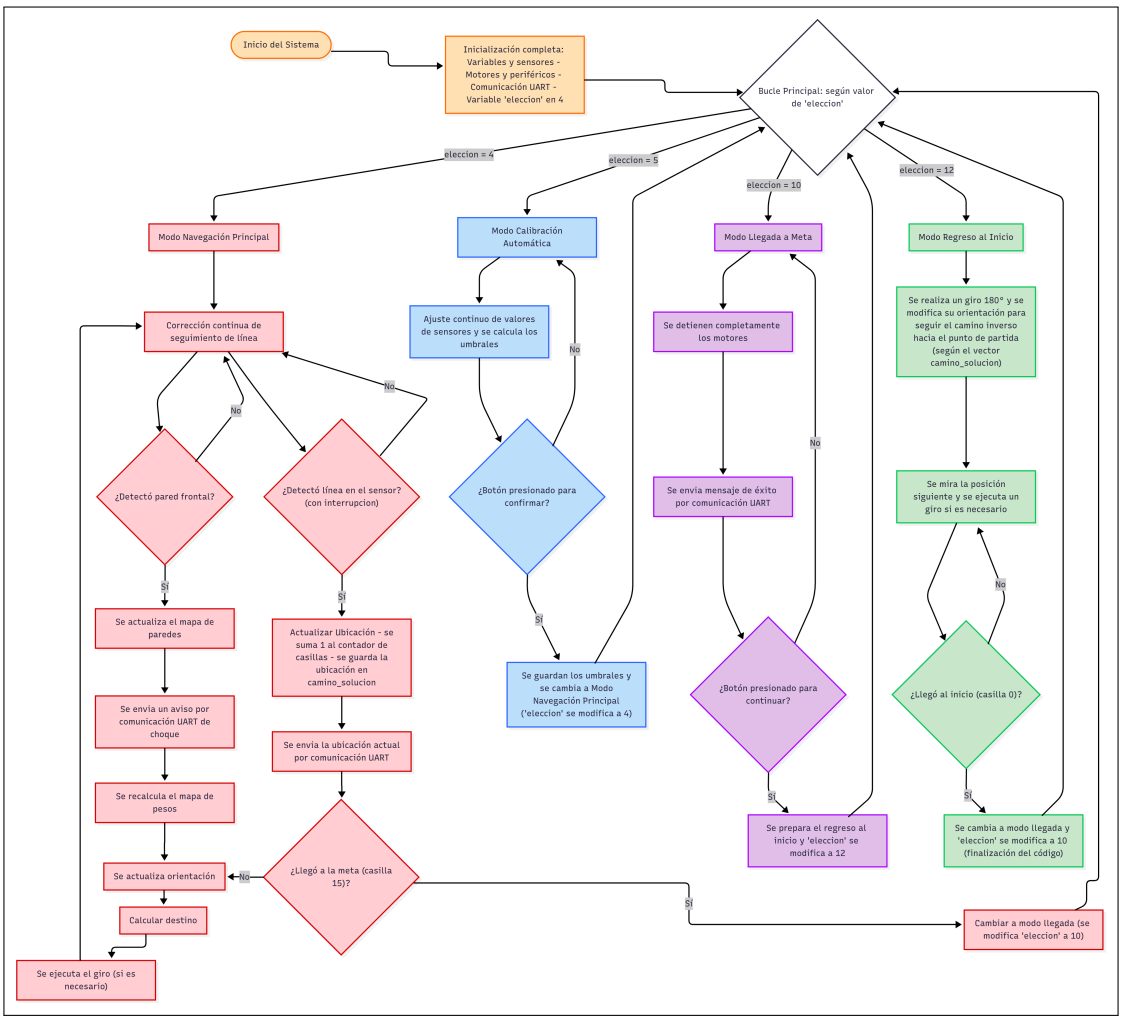


Figura 4. Diagrama de flujo de la segunda variante del algoritmo.

los sensores laterales. Por otro lado, si el control general decide que se debe realizar un giro para cambiar de casilla, este bloque ejecuta la acción.

- **Detección de obstáculos:** el robot emplea el sensor frontal para detectar muros y anotarlos en el mapa.
- **Detección de cuadrícula:** el sensor de línea registra el cruce de celdas y actualiza la posición actual en la matriz.
- **Control general y lógica de decisión:** implementación en C del *flood fill*, decide el próximo movimiento en función de la información disponible.

V. Conclusiones y trabajo futuro

Se integraron con éxito los sensores y motores del robot controlados por un microcontrolador STM32, permitiendo una respuesta en tiempo real para la navegación. A futuro, se plantea optimizar el código para reducir el tiempo de cálculo del algoritmo, y probar el sistema en laberintos de mayor tamaño.

La programación en C sobre la plataforma STM32 permitió un control preciso de los periféricos y una integración sencilla de los diferentes módulos de software. Sin embargo, a diferencia de soluciones en FPGA, la ejecución secuencial del microcontrolador puede requerir técnicas adicionales de optimización cuando la complejidad del entorno aumenta.

Referencias

[1] I. Elshamarka and A. B. S. Saman, "Design and implementation of a robot for maze-solving using flood-fill algorithm," *International Journal of Computer Applications*, vol. 56, no. 5, 2012.

[2] S. Tjiharjadi, "Design and implementation of flood fill and pledge algorithm for maze robot," *International Journal of Mechanical Engineering and Robotics Research*, 2019.

[3] Nadour and Cherroun, "Using flood-fill algorithms for an autonomous mobile robot maze navigation," *International Journal of System Assurance Engineering and Management*, 2022.

[4] STMicroelectronics, "L298N Dual Full Bridge Driver - Datasheet," https://www.handsontec.com/dataspecs/L298N%20Motor%20Driver.pdf, 2000, accessed: 2025-06-15.

[5] *User Manual: Discovery Kit with STM32F407VG MCU*, STMicroelectronics, May 2025, uM1472, Rev. 8. [Online]. Available: https://www.st.com/resource/en/user_manual/um1472-discovery-kit-with-stm32f407vg-mcu-stmicroelectronics.pdf

[6] Kai Morich, "Serial bluetooth terminal," https://play.google.com/store/apps/details?id=de.kai_morich.serial_bluetooth_terminal, 2025, aplicación móvil para Android, disponible en Google Play Store. [Online]. Available: https://play.google.com/store/apps/details?id=de.kai_morich.serial_bluetooth_terminal

[7] D. N. Mozo, "stm32-maze-solver," https://github.com/demianmozo/stm32-maze-solver, 2025.

[8] S. Garré, "Proyecto-laberinto," https://github.com/facundoiampietro/Proyecto_Laberinto.git, 2025.